1️⃣ Vision produit

Nom provisoire : *TaskSense* (ou ton propre nom 😎)

Concept :
Un **assistant personnel intelligent**, qui te permet de gérer tous tes projets et tâches dans ta vie quotidienne et professionnelle, avec une intégration multicanal (Email, WhatsApp, notifications) et une capacité à analyser ton flux d'informations pour te faire gagner du temps.

Objectif principal :

- Centraliser **toutes les tâches, projets et deadlines**
- Automatiser les **rappels et notifications intelligentes**
- Fournir des **analyses et résumés** de ton emploi du temps
- Faire de la **veille proactive** pour opportunités et nouvelles importantes

Problèmes que ça résout :

1. Trop de projets / idées dans la tête → tout est organisé et visualisé
2. Gestion manuelle des deadlines → notifications et rappels automatiques
3. Perte d'opportunités importantes → alertes pro personnalisées
4. Difficulté à suivre la productivité → résumés et analyses automatiques

---

2️⃣ Personas principaux

On définit **3 types d'utilisateurs** pour ton MVP / futur SaaS :

Persona 1 : L'étudiant / freelance (Toi au départ)

- **Nom :** Octave
- **Âge :** 18-30
- **Problème :** Trop de projets, deadlines dispersées, notifications dispersées entre mail et WhatsApp
- **Besoins :**
  - Tâches organisées avec étapes
  - Deadlines visibles et rappels automatiques

    - Analyse de productivité quotidienne / hebdomadaire
    - Notifications sur WhatsApp + mail
- **Objectif :** Gagner du temps, rester organisé, ne rien oublier

---

Persona 2 : Le professionnel multitâches

- **Nom :** Alice
- **Âge :** 25-40
- **Problème :** Emails et messages non triés, calendrier saturé, opportunités manquées
- **Besoins :**
    - Résumé intelligent des emails / messages importants
    - Alertes pour opportunités professionnelles
    - Intégration avec Google Calendar et outils professionnels
- **Objectif :** Maximiser efficacité, automatiser la gestion des tâches

---

Persona 3 : Le manager / petite équipe

- **Nom :** Kofi
- **Âge :** 30-50
- **Problème :** Suivi des projets de l'équipe, deadlines et notifications dispersées
- **Besoins :**
    - Assignation de tâches avec deadlines
    - Rappels automatiques pour les membres de l'équipe
    - Résumé et suivi de performance
- **Objectif :** Suivi simplifié, gain de temps dans le management

---

3️⃣ Fonctionnalités clés identifiées pour la vision produit

1. **Gestion de projets / tâches**
    - Projets → étapes → tâches
    - Deadlines → rappels automatiques
2. **Notifications multicanal**
    - Email, Google Calendar, WhatsApp, push notifications
3. **Analyse et résumé**
    - Productivité quotidienne / hebdomadaire

    - Priorisation intelligente des tâches
4. **Veille et opportunités**
    - Web scraping ciblé
    - Analyse emails entrants
    - Alertes pour opportunités importantes
5. **Intégrations**
    - APIs Gmail, Google Calendar
    - WhatsApp Business / Twilio
    - n8n pour orchestrer les workflows

✅ Résultat : une **vision claire et centrée utilisateur**, qui nous servira de guide pour **l'architecture technique (B)** et le MVP (C).

🏗️ Architecture Technique : TaskSense

1️⃣ Vue d'ensemble

```
[ Frontend Web / Mobile ]
        │
        ▼
[ Backend (Node.js / NestJS) ]
        │
        ├──> Base de données (PostgreSQL)
        ├──> Cache / Jobs (Redis + BullMQ)
        ├──> IA (OpenAI / LLM local)
        └──> n8n (Orchestration workflows)
                │
                ├──> Email API (Gmail, Outlook)
                ├──> Calendar API (Google Calendar)
                ├──> WhatsApp API (Twilio / 360dialog)
                └──> Web Scraping / Veille
```

2️⃣ Frontend

Objectif : Interface utilisateur simple, intuitive et interactive.

- **Tech :** React.js / Next.js (Web) → possibilité React Native (Mobile) plus tard
- **Fonctionnalités principales :**
  - Dashboard projets & tâches
  - Calendrier intégré
  - Notifications push (via service worker / Firebase)
  - Analyses et résumés visuels
  - Interface configuration workflows (intégration n8n)

💡 **Astuce IA :** Cursor AI peut générer rapidement composants, CRUD, pages et formulaires.

3️⃣ Backend

Objectif : Logiciel central qui gère les données, règles métier et communication avec IA et n8n.

- **Tech :** Node.js + NestJS (ou Express pour MVP simple)
- **Rôles principaux :**
  - Gestion des projets / tâches / deadlines
  - Authentification + rôles (JWT / OAuth2)
  - API REST pour frontend
  - Communication avec n8n (webhooks / API)
  - Scheduling / jobs → BullMQ + Redis
  - Intégration IA pour priorisation, résumé, analyse
- **Endpoints principaux :**
  - `/projects` → CRUD projets
  - `/tasks` → CRUD tâches + étapes
  - `/notifications` → gérer alertes
  - `/analytics` → résumés hebdo / jour
  - `/integrations` → connecter Gmail, WhatsApp, Calendar

4️⃣ Base de données

Objectif : Stocker toutes les données projets / tâches / utilisateurs / logs de notifications

- **Tech :** PostgreSQL
- **Tables principales :**
  1. `users` → id, nom, email, rôle, config notifications
  2. `projects` → id, nom, description, user_id, deadlines

3. `tasks` → id, projet_id, titre, description, deadline, status, priorité
4. `steps` → id, task_id, titre, status, ordre
5. `notifications` → id, user_id, type, channel, status, datetime
6. `analytics` → user_id, date, tâches complétées, score productivité
7. `integrations` → user_id, type (email, whatsapp), token

💡 Je peux te faire **un schéma visuel de la DB** si tu veux.

5️⃣ Jobs & Scheduler

- **BullMQ + Redis** pour gérer :
  - Rappels tâches
  - Deadlines
  - Webhooks vers n8n
- Permet de **déléguer les notifications et automatisations** sans bloquer le backend

---

6️⃣ IA / Intelligence

- **Objectif :**
  1. Priorisation automatique des tâches
  2. Résumés journaliers / hebdo
  3. Suggestions / alertes
  4. Analyse emails entrants ou contenus scraping
- **Tech :** OpenAI API (ou LLM local pour certaines tâches)
- **Workflow typique :**
  1. Backend → envoie contexte tâches / emails à l'IA
  2. IA → renvoie priorité / résumé / alertes
  3. Backend → stocke résultat + déclenche notifications via n8n

---

7️⃣ n8n — Orchestration / Automation

- **Rôle :**
  1. Connecter le backend aux **APIs externes**
  2. Gérer workflows multicanal
  3. Notifications via Email / WhatsApp / Push
  4. Web scraping / veille simplifiée

- **Exemple workflow :**
    1. Backend envoie webhook → n8n
    2. n8n récupère emails ou données web
    3. Analyse IA si nécessaire
    4. Notification envoyée sur les canaux configurés

8️⃣ Notifications Multicanal

- **WhatsApp :** Twilio / 360dialog → messages programmés
- **Email :** Gmail / Outlook API
- **Push :** service worker / Firebase

Le backend ne gère plus directement ces notifications, n8n orchestre tout.

9️⃣ Workflow global (Résumé rapide)

Utilisateur crée tâche → Backend enregistre →
BullMQ planifie job → Webhook n8n → n8n envoie notification →
IA analyse résumé / priorisation → Résultat stocké → Dashboard Frontend affiche

---

💡 **Point important pour “viber avec l’IA” :**

- Backend + DB = cœur que tu dois comprendre
- Frontend + n8n = canaux de vibe (tu peux générer vite avec l’IA)
- IA = copilote pour productivité + résumé

🏁 MVP TaskSense — Features exactes

1️⃣ Gestion de projets et tâches (Cœur du produit)

- Créer / éditer / supprimer **projets**
- Créer / éditer / supprimer **tâches** dans un projet
- Définir **étapes** pour chaque tâche
- Ajouter **deadlines**
- Voir les tâches **complètes et incomplètes**
- Dashboard simple avec **liste des tâches par projet / jour / semaine**
- Marquer une tâche ou étape comme **terminée**

2️⃣ Notifications et rappels (via n8n)

- Notification **Email** pour deadlines importantes
- Notification **Push Web** pour rappel à l'approche d'une deadline
- **Job Scheduler** avec BullMQ + Redis pour automatiser rappels
- ⚠️ WhatsApp et autres canaux → option **phase suivante**, mais préparer le système pour l'ajouter facilement

---

3️⃣ Analyse et résumé (IA léger)

- Résumé journalier des tâches à faire
- Résumé hebdomadaire : tâches complétées / en retard / prioritaires
- Suggestion automatique de **priorités pour la journée**

L'IA peut être **simple au départ**, juste résumé / tri prioritaire. Complexe plus tard.

4️⃣ Intégration Email / Calendar (API)

- Synchroniser les **deadlines des tâches** avec Google Calendar
- Créer automatiquement les événements pour chaque deadline
- Lire les emails pour **détecter les tâches importantes** (option avancée plus tard)

5️⃣ Backend / Frontend MVP

- Backend : Node.js + NestJS ou Express
- Frontend : React / Next.js
- Authentification simple : Email + mot de passe (JWT)
- Interface Dashboard + liste projets / tâches / étapes + notifications

---

6️⃣ Préparation pour extensions (future vibe)

Même si pas utilisé maintenant, prévoir :

- n8n pour WhatsApp / Push mobile / web scraping
- Tables DB pour analytics, intégrations, notifications
- IA pour priorisation complexe et résumé intelligent

✅ Résultat attendu du MVP

- **Tu peux l'utiliser pour organiser ta journée et tes projets**
- Les notifications te rappellent ce qu'il faut faire
- Dashboard simple pour visualiser ta charge de travail
- Base solide pour ajouter IA, WhatsApp, scraping plus tard

🛠️ Stack complète recommandée

Couche	Technologie / Outil	Rôle
Frontend	React.js / Next.js	Dashboard web, création projets/tâches, affichage notifications
Backend	Node.js + NestJS (ou Express pour MVP)	API REST, logique métier, gestion projets/tâches, auth
Base de données	PostgreSQL	Stockage projets, tâches, étapes, utilisateurs, notifications, analytics
Cache & Jobs	Redis + BullMQ	Scheduler pour rappels et notifications
Orchestration workflows	n8n	Automatisation Email / WhatsApp / Push / Calendar
IA / NLP	OpenAI API (ou LLM local)	Résumés, priorisation tâches, alertes importantes
Notifications	Gmail API, Google Calendar API, Twilio / 360dialog (WhatsApp), Service Worker / Firebase Push	Multicanal
Versioning / DevOps	Git + GitHub / GitLab	Gestion code

Hébergement	VPS / Cloud (ex : Render, Railway, OVH)	Backend + n8n + base PostgreSQL

🗄️ Schéma Base de données (simplifié mais complet pour MVP)

users

---

id (PK)

name

email

password_hash

role

notification_settings

projects

---

id (PK)

user_id (FK → users.id)

title

description

deadline

created_at

updated_at

tasks

---

id (PK)

project_id (FK → projects.id)

title

description

deadline

priority

status (todo, in-progress, done)

created_at

updated_at

steps

---

id (PK)

task_id (FK → tasks.id)

title

status (todo, done)

order

created_at

updated_at

notifications

---

id (PK)

user_id (FK → users.id)

type (email, push, whatsapp)

status (pending, sent)

message

scheduled_at

sent_at

analytics

---

id (PK)

user_id (FK → users.id)

date

tasks_completed

tasks_pending

productivity_score

integrations

---

id (PK)

user_id (FK → users.id)

type (email, whatsapp, calendar)

token

config_json

created_at

updated_at

💡 Notes importantes pour ton MVP

1. **n8n** s'intègre facilement via la table `integrations` et les webhooks.
2. **Redis + BullMQ** s'occupe du scheduling → pas besoin de poller la DB en permanence.
3. **IA** : pour commencer, tu peux juste utiliser les tables `tasks` et `analytics` pour générer priorités et résumés.
4. Le **frontend** doit être modulable pour ajouter push / WhatsApp / web scraping plus tard.

JOUR 1

Parfait ! 🎯 Aujourd'hui on se concentre **sur les fondations** : structure du projet, dépendances, clés API… tout pour que le MVP soit prêt à "viber" ensuite.

Voici **le plan concret étape par étape** pour la session d'aujourd'hui :

🏗️ Phase Fondations

1️⃣ Créer l'arborescence du projet

Backend (Node.js / NestJS ou Express)

```
task-sense-backend/
│
├── src/
│   ├── controllers/      # endpoints
│   ├── services/         # logique métier
│   ├── models/           # définitions DB (ORM)
│   ├── jobs/             # bullMQ jobs / cron
│   ├── utils/            # helpers
│   └── app.js / main.js  # point d'entrée
│
├── config/               # fichiers de configuration (DB, API keys)
├── package.json
├── .env                  # secrets et clés API
├── tsconfig.json (si TS)
└── README.md
```

Frontend (React / Next.js)

```
task-sense-frontend/
│
├── src/
│   ├── components/
│   ├── pages/
│   ├── services/         # appels API backend
│   ├── hooks/
│   └── utils/
│
├── package.json
├── .env.local            # secrets API pour frontend si besoin
└── README.md
```

💡 Astuce : tu peux créer ces dossiers **avec un simple script bash ou Node** pour gagner du temps.

2️⃣ Installer les dépendances

Backend (Express minimal / Node.js)

```
npm init -y
npm install express dotenv pg sequelize bullmq ioredis axios
npm install --save-dev nodemon
```

Frontend (React / Next.js minimal)

```
npx create-next-app@latest task-sense-frontend
cd task-sense-frontend
npm install axios react-query
```

> Pour l'instant, on installe **juste l'essentiel**. BullMQ + Redis pour scheduler, Axios pour appels API.

3️⃣ Ajouter les fichiers de configuration

Backend `.env`

```
PORT=3000
DB_HOST=localhost
DB_USER=postgres
DB_PASSWORD=yourpassword
DB_NAME=tasksense
REDIS_HOST=localhost
REDIS_PORT=6379
OPENAI_API_KEY=your_openai_key
GMAIL_CLIENT_ID=xxxx
GMAIL_CLIENT_SECRET=xxxx
TWILIO_SID=xxxx
TWILIO_TOKEN=xxxx
```

Frontend `.env.local` (si besoin pour API frontend) :

NEXT_PUBLIC_API_URL=http://localhost:3000/api

4️⃣ Configurer la base de données

- Crée ta **DB PostgreSQL** `tasksense`
- Connecte-la avec Sequelize (ou Prisma si tu préfères)
- Crée les **models / tables** : `users`, `projects`, `tasks`, `steps`, `notifications`, `analytics`, `integrations`

5️⃣ Configurer BullMQ + Redis

- Installer Redis localement ou via Docker
- Créer un **worker simple** pour tester un job de notification

```
const { Queue, Worker } = require("bullmq");
const connection = { host: "localhost", port: 6379 };

const taskQueue = new Queue("tasks", { connection });

const worker = new Worker(
  "tasks",
  async job => {
    console.log("Job exécuté :", job.name, job.data);
  },
  { connection }
);
```

6️⃣ Clés API et intégrations

- **OpenAI** → génération résumés / priorisation
- **Gmail / Google Calendar** → notifications et création d'événements
- **Twilio / 360dialog** → WhatsApp notifications
- **n8n** → orchestrer les workflows

Tu peux créer ces comptes maintenant et stocker les clés dans `.env`.

7️⃣ Test rapide

- Lancer backend : `nodemon src/app.js`
- Lancer frontend : `npm run dev`
- Tester une route simple `/ping` → doit répondre `"pong"`
- Tester un job BullMQ → console log doit afficher le job

✅ Résultat attendu **à la fin de la session** :

1. Projet backend + frontend créé
2. Arborescence complète et prête
3. Dépendances installées
4. Base PostgreSQL connectée
5. BullMQ + Redis prêt à scheduler des jobs
6. Clés API ajoutées dans `.env`
7. Premier test ping et job fonctionnel